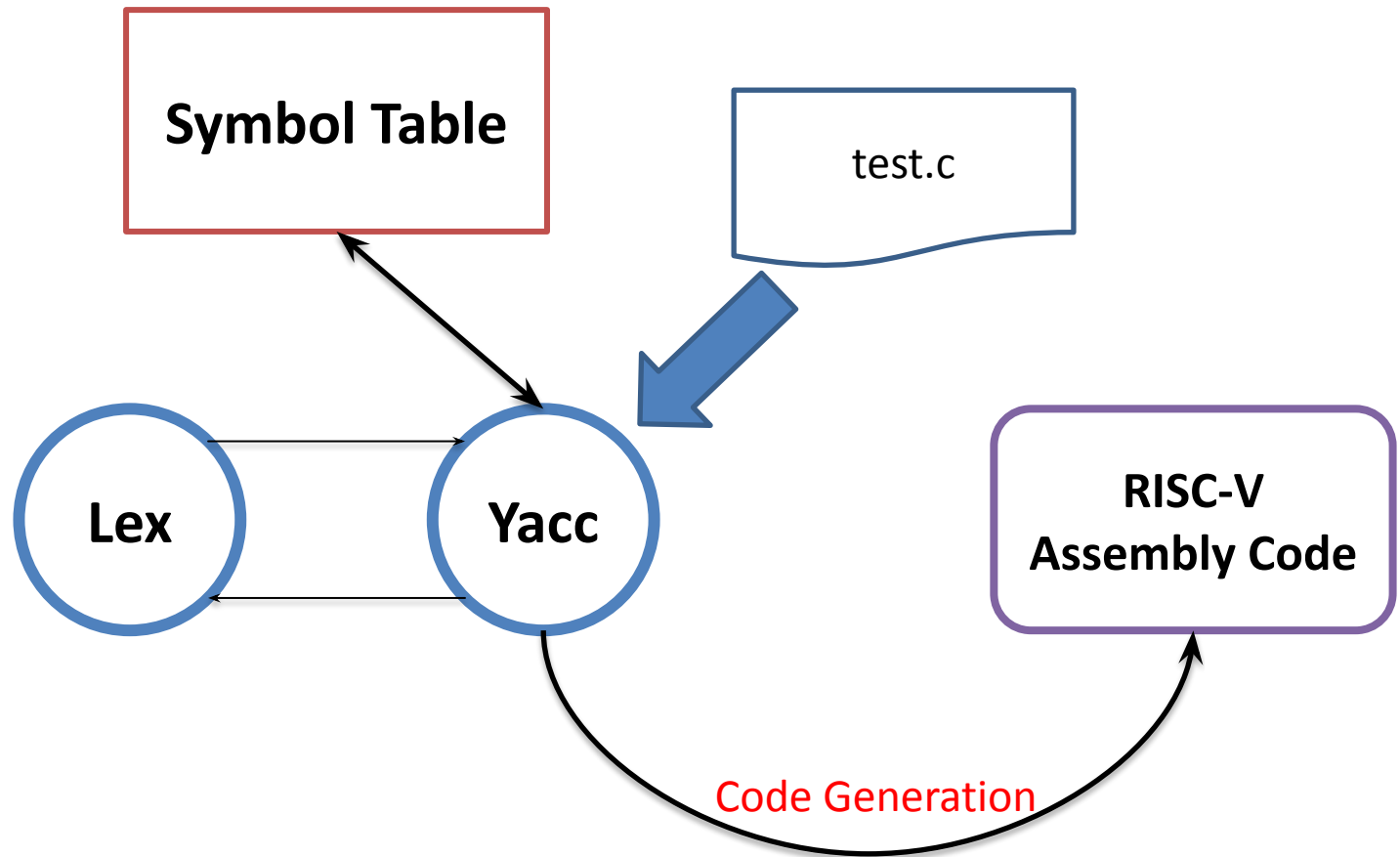


CS340400 Compiler Design

Homework 3

Submission Deadline: 2026/06/19 18:00

HW3 Architecture



Hints on Implementation

- Symbol Table
- Stack
- Generate Assembly Code

Symbol Table

- A table which keeps the information of symbols
 - E.g. scope, type, memory location, parameters, ...
- When a symbol (variable/function) declaration is encountered, store the information of the symbol into the symbol table
- When a symbol is accessed later on, read the symbol table to find out how to access the symbol

Stack

- A process has a stack memory allocated for it
- Program can allocate memory on stack for variables
- Program can save temporary results on stack

Generate Assembly Code

```
/* parser.y */
/* this is just an example, and it's incomplete */
/* but it does provide a hint on how to use stack */
...
expr: expr '+' expr {
    ...
    fprintf(f_asm, "    lw t0, %d(sp)\n", $1->stack_offset);
    fprintf(f_asm, "    lw t1, %d(sp)\n", $3->stack_offset);
    fprintf(f_asm, "    add t0, t0, t1\n");
    ...
};
...
```

Note on Generating Code for HW

- We use 32-bit RISC-V, which means you should use ``lw`` and ``sw`` with 4-byte memory space to load/save registers

Execute Assembly on RISC-V Spike Simulator

Prerequisite

- Have access to "spike" and "pk" on TA's server
- Have a piece of assembly code to execute
- Have access to the "assembly" (Spike version) sample project provided by TAs

Execute Assembly Code

1. Replace content of "codegen.S" with your assembly code
 - Your codegen binary should be able to produce the whole content of "codegen.S" without any manual tweak
2. Compile your program with
``riscv64-unknown-elf-gcc main.c codegen.S``
3. Execute your program with ``spike pk a.out``

What Happens in the Background

1. In main.c, ``void delay(int)`` and ``void digitalWrite(int, int)``, so you can use them in your "codegen.S"
2. In main.c, ``void codegen()`` is declared but not defined. Your "codegen.S" should define it
3. In main.c, the main function calls ``void codegen()``, which should be your generated program

HW3 Specification

Format Rule

- At least, a function ``void codegen();`` would be present in the input file
 - Both the declaration and definition
 - This is the entry point of the generated program
- For each function, your codegen binary should be able to compile it into assembly codes with the following format:

```
```\n.global <func_name>\n<func_name>:\n    (assembly implementation)\n```
```

# Special Function

- Function invocation codegen is not mandatory, except for ``digitalWrite`` and ``delay``
  - Students can either implement generic function invocation codegen, or tackle these 2 functions specifically with Lex and Yacc modification

# Special Function (cont.)

- `digitalWrite(pin, value)`
  - `pin`: an integer
  - `value`: `HIGH` or `LOW`
    - `HIGH` is `1`; `LOW` is `0`
  - Writes a HIGH/LOW signal to the specified pin
  - <https://www.arduino.cc/reference/en/language/functions/digital-io/digitalwrite/>

# Special Functions (cont.)

- ``delay (ms) ``
  - ``ms``: an integer
  - Sleep for the specified time
  - <https://www.arduino.cc/reference/en/language/functions/time/delay/>



# Special Functions (cont.)

- Prepare arguments in ``a0`, `a1`, ...`
- Call ``jal ra, <func_name>`` to jump to function and set return address in ``ra``
- E.g. to invoke ``delay(1000);``:

```
li a0, 1000
jal ra, delay
```

- Note that you should save caller-saved registers onto stack before invocation and restore them afterwards
  - RISC-V Calling Convention Reference:  
<<https://riscv.org/wp-content/uploads/2015/01/riscv-calling.pdf>>

# Grading Policies

# Misc. Rule

- No static/manual optimization. We want to see how your codegen binary handles the program structure
  - E.g. no constant propagation, no constant folding, no dead code elimination, ...
- Token/Syntax requirement is the same as HW1/2

# Testcase "Basic"

- Grade Lv.: E
- #testcases: 1
- Description
  - Your compiler should handle the basic structures of testcases correctly, including but not limited to ``codegen`` function declaration/definition, and invocations of special functions

# Testcase "Arithmetic Expression"

- Grade Lv.: D
- #testcases: 2
- Description
  - Your compiler should handle arithmetic expressions composed of ``+``, ``-``, ``*``, ``/`` and parentheses correctly

# Testcase "Pointer"

- Grade Lv.: C
- #testcases: 2
- Description
  - Your compiler should handle single-level pointers correctly

# Testcase "Jump"

- Grade Lv.: B
- #testcases: 2
- Description
  - Your compiler should handle branching/loop statements correctly
  - Your compiler should handle 1D array correctly

# Testcase "Function"

- Grade Lv.: A
- #testcases: 2
- Description
  - Your compiler should handle generic function invocations with the RISC-V calling convention correctly
  - RISC-V Calling Convention Reference:  
<<https://riscv.org/wp-content/uploads/2015/01/riscv-calling.pdf>>



# Grading

- E: 50pt
- D: 10pt (5pt each testcase)
- C: 10pt (5pt each testcase)
- B: 10pt (5pt each testcase)
- A: 10pt (5pt each testcase)
- Hidden: 10pt (similar to A)

# Submission

- Submit on eeclass
  - Deadline: 06/19 18:00
  - scanner.l
  - parser.y
  - Assume all files are in the same directory.
    - We will run “make codegen” followed by “./codegen < {testcase}” to test your program.
    - After running “./codegen < {testcase}”, the output must be written to a file named “codegen.S”.